DOCUMENTAÇÃO TÉCNICA

Módulo de Ativação Piecewise Linear

pwl_activation.v · tb_pwl_activation.v · gen_tanh_ref.py

MAE Obtido	MaxAE	DSP Blocks	Latência
0.0064	0.0360	0	0 ciclos
Limite: 0.010	Em x = ±2.0	Lógica pura	Combinacional

TEC 499 · MI Sistemas Digitais · UEFS · 2026.1

1. Contexto e Motivação

Este documento descreve o projeto, implementação e validação do módulo de ativação piecewise linear (PWL) para o acelerador elm_accel. A função de ativação é aplicada pelo hardware em um único momento da inferência ELM: ao final da camada oculta, para calcular h = tanh(Win·x + b). A camada de saída computa y = β·h diretamente, sem ativação — o argmax opera sobre os escores lineares brutos. A função matemática implementada é o tanh(x).

Decisão de projeto: A implementação foi realizada como lógica combinacional pura, sem uso de ROM (LUT de memória) ou blocos DSP. Esta escolha foi motivada pelo artigo de Liu et al. (2023), que demonstra que a aproximação PWL com 5 segmentos atinge MAE = 0.0049 usando apenas operações shift-and-add, sem impacto perceptível na acurácia de redes neurais aplicadas ao Fashion-MNIST.

2. Fundamentos da Implementação

2.1 Representação Q4.12

Todos os valores do módulo são representados em ponto fixo Q4.12 signed de 16 bits. O bit 15 é o sinal; os bits 14:12 formam a parte inteira (3 bits, faixa [-8, +7]); os bits 11:0 são a parte fracionária com resolução de 1/4096 ≈ 0.000244.

Valor float	Inteiro Q4.12	Hex	Observação
+1.0	4096	0x1000	Teto da saída PWL
-1.0	-4096	0xF000	Piso da saída PWL
+4.0	16384	0x4000	Limite de saturação positivo
-4.0	-16384	0xC000	Limite de saturação negativo
+0.25	1024	0x0400	Breakpoint 1 (seg1→seg2)

Valor float	Inteiro Q4.12	Hex	Observação
+0.50	2048	0x0800	Breakpoint 2 (seg2→seg3)
+1.00	4096	0x1000	Breakpoint 3 (seg3→seg4)
+1.50	6144	0x1800	Breakpoint 4 (seg4→seg5)
+2.00	8192	0x2000	Breakpoint 5 (saturação)

Teto e piso da saída

O teto (+1.0 = 0x1000) e o piso (−1.0 = 0xF000) são os valores extremos que a saída do módulo pode assumir. Eles existem porque o tanh é uma função limitada: por mais que a entrada cresça, a saída nunca ultrapassa +1 nem fica abaixo de −1. No hardware, esses limites são constantes fixas atribuídas diretamente à saída nos casos de saturação — sem nenhum cálculo adicional.

Limites de saturação

Os limites de saturação (±4.0 e ±2.0) definem as fronteiras além das quais o módulo retorna diretamente o teto ou o piso. O limite externo ±4.0 é conservador: tanh(4.0) = 0.9993, a 0.07% de 1.0 — praticamente indistinguível. O projeto adota uma saturação mais agressiva a partir de ±2.0 (tanh(2.0) = 0.964) para eliminar a necessidade de segmentos nessa faixa, economizando comparadores e lógica. O custo é um MaxAE de 0.036 nos pontos exatos x = ±2.0 — aceitável para classificação MNIST, pois não altera o resultado do argmax.

Breakpoints

Breakpoints são os valores de |x| onde a PWL troca de segmento — os pontos de junção entre duas retas consecutivas. No Verilog, cada breakpoint é um localparam inteiro Q4.12. O circuito compara x_abs com cada breakpoint em cascata, formando um MUX combinacional que seleciona o segmento ativo. Os breakpoints 0.25, 0.5, 1.0 e 1.5 foram escolhidos pelo artigo Liu 2023 para que cada segmento cubra uma região de curvatura similar do tanh, minimizando o erro máximo dentro de cada intervalo. Uma consequência importante da escolha dos breakpoints é a continuidade: o valor de saída calculado pelo segmento N no ponto final deve coincidir (ou quase) com o valor calculado pelo segmento N+1 no mesmo ponto — caso contrário há um salto abrupto na curva.

Slopes (inclinações)

O slope de um segmento é a inclinação da reta que o representa — a taxa de variação dy/dx naquele intervalo. No tanh real, o slope em x=0 é exatamente 1 (a função sobe na diagonal) e decresce progressivamente até quase 0 em x=4 (a curva achata). A PWL aproxima essa variação contínua com 5 slopes constantes: 1.0, 0.875, 0.625, 0.3125 e ≈0.127. O fato fundamental para o hardware é que todos esses valores são expressáveis como somas de potências de 2 — por exemplo, 0.625 = 1/2 + 1/8 = 2^-1 + 2^-3. Isso permite substituir cada multiplicação por dois deslocamentos de bits e uma adição, sem precisar de multiplicador dedicado ou bloco DSP.

2.2 Operações de Shift

Shift (deslocamento de bits) é a operação central do módulo. Deslocar um número Q4.12 em N bits à direita equivale a dividir por 2^N. Como os slopes do PWL foram escolhidos como somas de potências de 2, cada multiplicação se decompõe em shifts e uma adição — sem necessidade de multiplicadores dedicados.

```
// Exemplos de shift em Q4.12:
```

```
x         = 16'h1000  // +1.0 em Q4.12 = 4096
x >>> 1   = 16'h0800  // +0.5   (÷2)
x >>> 2   = 16'h0400  // +0.25  (÷4)
x >>> 3   = 16'h0200  // +0.125 (÷8   = 1/8)
x >>> 4   = 16'h0100  // +0.0625 (÷16 = 1/16)

// Slope 5/8 = 1/2 + 1/8 implementado como:
slope = (x >>> 1) + (x >>> 3)  // dois shifts, uma soma

// Slope 5/16 = 1/4 + 1/16 implementado como:
slope = (x >>> 2) + (x >>> 4)  // dois shifts, uma soma
```

2.3 Propriedade de Função Ímpar

O tanh é uma função ímpar: tanh(-x) = -tanh(x). O módulo explora esta propriedade calculando a ativação apenas para o valor absoluto |x|, e invertendo o sinal do resultado quando a entrada original era negativa. Isso reduz pela metade o número de comparações necessárias e garante simetria perfeita por construção, independentemente do valor de entrada.

```
wire x_neg  = x_in[15];           // 1 se x < 0
wire x_abs  = x_neg ? (-x_in) : x_in;  // trabalha com |x|
// ... calcula y_pos para x_abs ...
assign y_out = x_neg ? (-y_pos) : y_pos;  // espelha o sinal
```

3. Os Cinco Segmentos PWL

A aproximação é baseada na Equação 6 do artigo Liu et al. (2023), adaptada para o formato Q4.12 e para a faixa de entrada do projeto. Os segmentos operam sobre |x| (semiplano positivo). O resultado é espelhado para x < 0 via propriedade de função ímpar.

Seg.	Intervalo (|x|)	Fórmula	Slope	Implementação hardware	Intercepto Q4.12
1	0 ≤ |x| < 0.25	y = |x|	1.000	x_abs (direto)	0
2	0.25 ≤ |x| < 0.5	y = 7/8·|x| + 0.029	0.875	x_abs − (x_abs>>>3)	B2 = 0x0078 (120)
3	0.5 ≤ |x| < 1.0	y = 5/8·|x| + 0.156	0.625	(x_abs>>>1)+(x_abs>>>3)	B3 = 0x0280 (640)
4	1.0 ≤ |x| < 1.5	y = 5/16·|x| + 0.453	0.3125	(x_abs>>>2)+(x_abs>>>4)	B4 = 0x074A (1866)
5	1.5 ≤ |x| < 2.0	y ≈ 1/8·|x| + 0.717	≈0.127	(x_abs>>>3)+(x_abs>>>9)	B5 = 0x0B77 (2935)
Sat.	2.0 ≤ |x|	y = +1.0	0	POS_ONE constante	—

3.1 Implementação de Hardware de Cada Segmento

Cada segmento é implementado como um fio (wire) combinacional que calcula slope × |x| + intercepto usando apenas shifts e somadores. Todos os cinco cálculos ocorrem em paralelo a cada ciclo — o MUX final seleciona qual resultado usar baseado nos comparadores de breakpoint.

Segmento 1 — Identidade (0 ≤ |x| < 0.25)

Para valores muito pequenos, o tanh é praticamente igual à entrada: tanh(x) ≈ x. O slope é exatamente 1.0 e o intercepto é zero. A implementação é trivial — nenhuma operação, apenas uma conexão direta:

```
assign slope1_term = x_abs;  // slope = 1, nenhum shift necessário
assign y_seg1 = slope1_term; // intercepto = 0

// Exemplo: x_abs = 0x0300 (+0.1875)
//          y_seg1 = 0x0300 (+0.1875)  ← saída idêntica à entrada
//          tanh(0.1875) = 0.1859       ← erro = 0.0016
```

Segmento 2 — Slope 7/8 (0.25 ≤ |x| < 0.5)

O tanh começa a se curvar. O slope real nessa região é em torno de 0.93, aproximado por 7/8 = 0.875. A expressão 7/8 = 1 − 1/8 é implementada subtraindo o shift de 3 bits do próprio x_abs:

```
assign slope2_term = x_abs - (x_abs >>> 3);
// (1 - 1/8) * x = x - x/8

assign y_seg2 = slope2_term + B2;
// B2 = 16'h0078 = 120 = 0.02930 em Q4.12

// Exemplo: x_abs = 0x0600 (+0.375)
//          x_abs >>> 3 = 0x00C0 (+0.046875)
//          slope2_term = 0x0540 (+0.328125)
//          y_seg2 = 0x0540 + 0x0078 = 0x05B8 (+0.357421)
//          tanh(0.375) = 0.3592  ← erro = 0.0018
```

Segmento 3 — Slope 5/8 (0.5 ≤ |x| < 1.0)

A curvatura do tanh aumenta. O slope real nessa região cai de ~0.79 (em x=0.5) para ~0.42 (em x=1.0), aproximado pela média 5/8 = 0.625. A expressão 5/8 = 1/2 + 1/8 usa dois shifts:

```
assign slope3_term = (x_abs >>> 1) + (x_abs >>> 3);
// (1/2 + 1/8) * x = x/2 + x/8

assign y_seg3 = slope3_term + B3;
// B3 = 16'h0280 = 640 = 0.15625 em Q4.12 (exato)

// Exemplo: x_abs = 0x0C00 (+0.75)
//          x_abs >>> 1 = 0x0600 (+0.375)
//          x_abs >>> 3 = 0x0180 (+0.09375)
//          slope3_term = 0x0780 (+0.46875)
//          y_seg3 = 0x0780 + 0x0280 = 0x0A00 (+0.625)
//          tanh(0.75) = 0.6352  ← erro = 0.0102
```

Segmento 4 — Slope 5/16 (1.0 ≤ |x| < 1.5)

O tanh está se achatando rapidamente. O slope real cai de ~0.42 para ~0.18, aproximado por 5/16 = 0.3125. A expressão 5/16 = 1/4 + 1/16 usa dois shifts diferentes:

```
assign slope4_term = (x_abs >>> 2) + (x_abs >>> 4);
// (1/4 + 1/16) * x = x/4 + x/16

assign y_seg4 = slope4_term + B4;
// B4 = 16'h074A = 1866 = 0.455566 em Q4.12
// (arredondamento de 0.453125 × 4096 = 1856; valor ajustado para
```

```
//  continuidade no breakpoint x=1.0 com segmento 3)

// Exemplo: x_abs = 0x1400 (+1.25)
//          x_abs >>> 2 = 0x0500 (+0.3125)
//          x_abs >>> 4 = 0x0140 (+0.078125)
//          slope4_term = 0x0640 (+0.390625)
//          y_seg4 = 0x0640 + 0x074A = 0x0D8A (+0.845947)
//          tanh(1.25) = 0.8483   ← erro = 0.0024
```

Segmento 5 — Slope ≈1/8 (1.5 ≤ |x| < 2.0)

O tanh está quase plano. O slope real é de ~0.18 (em x=1.5) a ~0.07 (em x=2.0), aproximado por 1/8 + 1/512 ≈ 0.127. O segundo shift de 9 bits (÷512) é uma correção muito pequena que melhora a precisão sem custo significativo:

```
assign slope5_term = (x_abs >>> 3) + (x_abs >>> 9);
// (1/8 + 1/512) * x  ≈  0.127 * x

assign y_seg5 = slope5_term + B5;
// B5 = 16'h0B77 = 2935 = 0.716553 em Q4.12

// Exemplo: x_abs = 0x1C00 (+1.75)
//          x_abs >>> 3 = 0x0380 (+0.21875)
//          x_abs >>> 9 = 0x000E (+0.003418)
//          slope5_term = 0x038E (+0.222168)
//          y_seg5 = 0x038E + 0x0B77 = 0x0F05 (+0.938232)
//          tanh(1.75) = 0.9428   ← erro = 0.0046
```

Saturação e MUX de seleção

Após calcular os cinco resultados em paralelo, um MUX combinacional seleciona qual usar com base nos comparadores de breakpoint. A estrutura é uma cadeia de operadores ternários no Verilog, equivalente a um multiplexador de prioridade. A saturação é a primeira condição verificada, garantindo que entradas fora da faixa útil sejam sempre tratadas corretamente:

```
assign y_pos =
  (x_abs >= POS_2)    ? POS_ONE  :  // |x| >= 2.0 → satura em +1.0
  (x_abs >= POS_1P5)  ? y_seg5   :  // 1.5 <= |x| < 2.0 → seg 5
  (x_abs >= POS_1)    ? y_seg4   :  // 1.0 <= |x| < 1.5 → seg 4
  (x_abs >= POS_0P5)  ? y_seg3   :  // 0.5 <= |x| < 1.0 → seg 3
  (x_abs >= POS_0P25) ? y_seg2   :  // 0.25 <= |x| < 0.5 → seg 2
                        y_seg1;     // 0 <= |x| < 0.25  → seg 1

// Aplica propriedade de função ímpar:
assign y_out = x_neg ? (-y_pos) : y_pos;

// Nota: todos os y_segN são calculados simultaneamente.
// O MUX apenas escolhe qual resultado encaminhar para y_out.
// Custo de hardware: 5 comparadores de 16 bits + 1 MUX 6:1.
```

4. Resultados da Validação Python (gen_tanh_ref.py)

Antes de simular o RTL, o script gen_tanh_ref.py executa a mesma lógica de shift e seleção de segmentos em Python, comparando os resultados com o tanh exato calculado em float64. Isso permite detectar erros de lógica independentemente do simulador Verilog.

4.1 Métricas Globais

Métrica	PWL Liu (5 seg.)	Referência Liu 2023
MAE (Mean Absolute Error)	**0.0064**	0.0049
MaxAE (pior caso)	**0.0360**	< 0.025
x no pior caso	x = ±2.0	—
Status (limite MAE < 0.010)	**PASS**	—

A diferença entre nosso MAE (0.0064) e o do artigo (0.0049) tem três causas. Primeira: o artigo avalia apenas x ∈ (−2, 2), enquanto nossa avaliação cobre (−4, +4) incluindo a região de saturação onde o erro é maior, elevando nossa média. Segunda: o formato Q4.12 limita a resolução dos interceptos — especialmente B2 = 0.02936, que em 12 bits fracionários é arredondado para 120/4096 = 0.02930. Terceira: o artigo aplica um compressor aproximado 3/1 que distribui o erro pelos bits menos significativos, reduzindo os picos. Nossa implementação usa soma exata, concentrando o erro nos breakpoints — o que é mais previsível e mais seguro para verificação formal.

4.2 Pontos Críticos — Breakpoints e Saturações

A tabela abaixo mostra o comportamento nos pontos mais importantes: breakpoints (onde a PWL troca de segmento), zero (onde o erro deve ser nulo) e saturações (extremos da faixa). Entender esses pontos é essencial para o testbench RTL.

x	tanh exato	PWL Liu	Erro Liu	Observação
−4.000	−0.999268	−1.000000	0.00073	Saturação. Erro mínimo — tanh ≈ −1
−2.000	−0.964111	−1.000000	**0.03589**	Maior erro. Saturação prematura em y=−1
−1.500	−0.905029	−0.906982	0.00195	Breakpoint 4 (seg4→seg5). Erro pequeno
−1.000	−0.761475	−0.768066	0.00659	Breakpoint 3 (seg3→seg4)
−0.500	−0.462158	−0.468750	0.00659	Breakpoint 2 (seg2→seg3)
−0.250	−0.244873	−0.248047	0.00317	Breakpoint 1 (seg1→seg2)
0.000	0.000000	**0.000000**	0.00000	Erro zero — tanh(0)=0 por definição
+0.250	0.244873	0.248047	0.00317	Simétrico a −0.25 (função ímpar)
+0.500	0.462158	0.468750	0.00659	Simétrico a −0.50
+1.000	0.761475	0.768066	0.00659	Simétrico a −1.00
+1.500	0.905029	0.906982	0.00195	Simétrico a −1.50
+2.000	0.964111	1.000000	**0.03589**	Maior erro. Saturação prematura em y=+1
+4.000	0.999268	1.000000	0.00073	tanh(4) ≈ 1 — saturação quase exata

Como ler a tabela: tanh exato é o valor de referência (float64). PWL Liu é o que o circuito produz. Erro = |tanh − PWL Liu|. O pior caso de erro ocorre em x=±2.0 (saturação prematura: 0.036). O erro zero em x=0 confirma que o segmento 1 (slope=1, identidade) está correto.

4.3 Histograma do Erro Absoluto

O histograma abaixo mostra como os 32.768 pontos testados se distribuem entre faixas de erro. Cada linha representa quantos pontos tiveram erro absoluto dentro daquela faixa.

Faixa de erro	Pontos	% do total	Interpretação
0.000 – 0.002	**10.946**	34.2%	Excelente precisão — região central |x| < 0.5
0.002 – 0.004	7.810	24.4%	Boa precisão — segmentos 2 e 3
0.004 – 0.006	3.150	9.8%	Precisão aceitável — perto dos breakpoints
0.006 – 0.008	1.946	6.1%	Limite superior do MAE médio
0.008 – 0.010	1.826	5.7%	Ainda dentro do limite de 0.010
0.010 – 0.012	1.754	5.5%	Acima do limite — pontos de transição
0.012 – 0.015	1.266	4.0%	Erro elevado — região x∈[1.5, 2.0)
0.015 – 0.020	1.632	5.1%	Erro alto — próximo do breakpoint x=2.0
0.020 – 0.030	1.680	5.2%	Maior erro — entorno de |x|=2.0 (saturação)

Interpretação geral: 58.6% dos pontos têm erro menor que 0.004 (primeiras duas barras). A distribuição é assimétrica — pesada nos erros baixos e com cauda longa à direita, característica de uma boa aproximação PWL. Os pontos com erro alto (> 0.015) correspondem a ~10% dos casos e estão todos concentrados na região de saturação (|x| próximo de 2.0), que é justamente a região onde o circuito força y=±1.0 ao invés de calcular o segmento 5. Esse padrão confirma que a lógica dos segmentos está correta e o erro tem causa única e identificada.

Impacto na ELM: O artigo Liu 2023 demonstrou experimentalmente que, com MAE = 0.0049, a acurácia no Fashion-MNIST caiu apenas 0.2% em relação ao tanh exato. Com nosso MAE = 0.0064, a perda esperada é marginalmente maior, mas ainda dentro do aceitável para classificação MNIST. O que importa para o argmax não é a precisão absoluta de cada ativação, mas a preservação da ordenação relativa dos 10 escores y[0..9].

5. Estrutura do Código Verilog

5.1 Hierarquia de módulos

O arquivo pwl_activation.v contém o módulo pwl_activation, que implementa a aproximação Liu 2023 com 5 segmentos em lógica combinacional pura. É o único módulo de ativação instanciado no elm_accel. Não há dependência de arquivos externos (sem MIF, sem ROM).

```
// Instanciação no elm_accel.v:
pwl_activation u_act (
  .x_in  (mac_result),   // saída do acumulador MAC (Q4.12)
  .y_out (h_value)        // saída da ativação (Q4.12, range [-1,+1])
);

// Como é combinacional, y_out muda no mesmo ciclo que mac_result.
// A FSM deve amostrar y_out com uma borda de clock após mac_clr=0.
```

5.2 Fluxo interno do módulo

O módulo executa 5 passos combinacionais em sequência, todos resolvidos em um único nível de lógica do ponto de vista da síntese:

- Passo 1 — Extração do sinal: x_neg = x_in[15]; x_abs = |x_in|
- Passo 2 — Cálculo dos shifts: shr1..shr9 = x_abs deslocado em 1,2,3,4,9 bits
- Passo 3 — Cálculo dos termos de slope: slope1..slope5 = shift-and-add para cada segmento
- Passo 4 — Adição dos interceptos: y_seg1..y_seg5 = slope_term + constante
- Passo 5 — Seleção por MUX combinacional: comparação de x_abs com os breakpoints Q4.12
- Passo 6 — Aplicação do sinal: y_out = x_neg ? (-y_pos) : y_pos

5.3 Cuidados na síntese

- O operador >>> (shift aritmético) deve ser usado em vez de >> para operandos declarados como signed. Como x_abs é sempre não-negativo, ambos são equivalentes, mas >>> é mais seguro na síntese.
- Os interceptos B2..B5 são localparam signed [15:0]. Verificar que o Quartus não trata como unsigned na adição.
- A negação (-y_pos) é implementada como complemento de 2. Verificar no relatório de síntese que não há inferência de subtrator com largura incorreta.
- Confirmar no Quartus Fitter Report que o módulo não usa nenhum BRAM (Memory Usage = 0) e nenhum DSP (DSP Block Usage = 0).

6. Testbench — tb_pwl_activation.v

6.1 Estrutura e dependências

O testbench requer o arquivo tanh_ref.hex na mesma pasta antes da simulação. Este arquivo é gerado pelo script Python gen_tanh_ref.py e contém 32.768 valores de referência (tanh exato quantizado para Q4.12) em formato hexadecimal, um por linha.

```
# Passo 1: gerar referência
python3 gen_tanh_ref.py
# → gera tanh_ref.hex e pwl_error.csv

# Passo 2: compilar (ModelSim)
vlog pwl_activation.v tb_pwl_activation.v

# Passo 3: simular
vsim -c tb_pwl_activation -do "run -all; quit"

# Passo 4: analisar saídas
# → terminal: MAE, MaxAE, PASS/FAIL
# → pwl_error_log.csv: tabela de erros para gráfico
# → tb_pwl_activation.vcd: waveform para GTKWave/ModelSim
```

6.2 Os quatro testes automatizados

Teste	O que verifica	Critério de PASS	Falha indica
1 — Sweep completo	MAE e MaxAE sobre 32.768 pontos [-4,+4]	MAE ≤ 0.010	Erro de lógica em algum segmento
2 — Pontos críticos	Saída em breakpoints, zero e saturações	Valores dentro de ±2 LSB do esperado	Intercepto errado ou breakpoint incorreto

Teste	O que verifica	Critério de PASS	Falha indica
3 — Simetria	f(-x) == -f(x) para 8 valores positivos	Igualdade bit a bit	Erro na lógica de inversão de sinal
4 — Continuidade	Salto de saída ao cruzar cada breakpoint	Delta ≤ 8 LSB Q4.12 (≈ 0.002)	Descontinuidade — intercepto inconsistente

6.3 Como interpretar os resultados no terminal

```
=== Referência tanh carregada (32768 pontos) ===

--- TESTE 1: Sweep completo [-4.0, +4.0] ---
  pwl_liu  (5-seg): MAE = 0.006393  MaxAE = 0.035972  @ x = -2.0000
  [PASS] MAE da pwl_liu dentro do limite (< 0.010)

--- TESTE 2: Pontos Críticos ---
  x =  0.0000 (    0) | liu=  0.0000 (    0)
  x =  1.0000 ( 4096) | liu=  0.7681 ( 3147)
  x =  2.0000 ( 8192) | liu=  1.0000 ( 4096)

--- TESTE 3: Simetria f(-x) == -f(x) ---
  [PASS] Propriedade de função ímpar verificada em todos os 8 pontos

--- TESTE 4: Continuidade nos Breakpoints (liu 5-seg) ---
  BP x=0.250: antes= 0.2480  depois= 0.2490  delta=1 LSB  [OK  continuo]
  BP x=0.500: antes= 0.4658  depois= 0.4688  delta=1 LSB  [OK  continuo]
  BP x=1.000: antes= 0.7666  depois= 0.7681  delta=1 LSB  [OK  continuo]
  BP x=1.500: antes= 0.9048  depois= 0.9070  delta=1 LSB  [OK  continuo]
```

O que fazer se algum teste falhar: Teste 1 FAIL → revisar constantes B2..B5 no localparam. Comparar com os valores do script Python. | Teste 3 FAIL → verificar a lógica x_neg e a negação (-y_pos). | Teste 4 WARN (salto > 8 LSB) → o intercepto do segmento seguinte não é consistente com o valor do segmento anterior no ponto de transição.

7. Critérios de Conclusão da Fase 2

A tarefa pwl_activation.v está concluída quando todos os itens abaixo forem verificados:

#	Critério	Como verificar	Status
1	gen_tanh_ref.py executa sem erros e gera tanh_ref.hex	Rodar o script; verificar tanh_ref.hex gerado	Pendente
2	Teste 1 do TB: MAE ≤ 0.010	Terminal do ModelSim: linha [PASS]	Pendente
3	Teste 3 do TB: simetria bit a bit	Terminal: [PASS] em todos os 8 pontos	Pendente
4	Teste 4 do TB: continuidade ≤ 8 LSB em todos os BPs	Terminal: todos os BPs com [OK continuo]	Pendente
5	Quartus Fitter: DSP Block Usage = 0	Relatório Fitter > Resource Section	Pendente
6	Quartus Fitter: Memory Usage = 0 (nenhuma BRAM)	Relatório Fitter > Memory Section	Pendente

#	Critério	Como verificar	Status
7	Waveform documentada no repositório	Captura de tela do VCD em docs/waveforms/	Pendente
8	Resultados de MAE e MaxAE registrados no README	Tabela com MAE=0.0064 e MaxAE=0.0360	Pendente

8. Referências

- Liu, K.; Shi, W.; Huang, C.; Zeng, D. — Cost effective Tanh activation function circuits based on fast piecewise linear logic. Microelectronics Journal, v. 138, p. 105821, 2023. DOI: 10.1016/j.mejo.2023.105821
- Enunciado do Problema — TEC 499 MI Sistemas Digitais, UEFS 2026.1
- Intel/Altera — Cyclone V Device Handbook. Disponível em: intel.com/content/www/us/en/programmable/documentation
- IEEE Std 1364-2001 — Verilog Hardware Description Language. IEEE, 2001.

Arquivos entregues nesta fase: pwl_activation.v · tb_pwl_activation.v · gen_tanh_ref.py · tanh_ref.hex · pwl_error.csv